



PyQuest

Information- & Aufgabenblatt

Kapitel 8: while-Schleifen

Wiederhole Code, solange eine Bedingung gilt – ohne ihn abzuschreiben.

while-Schleifen

Oft soll ein Programm **ähnliche Anweisungen mehrfach hintereinander** ausführen. Genau dafür gibt es **Schleifen**. Sie sparen Zeit, vermeiden Tippfehler und machen den Code besser lesbar.

Eine **while-Schleife** wiederholt einen Codeblock, **solange** eine Bedingung wahr ist. Sobald die Bedingung False wird, hört die Schleife auf und das Programm läuft normal weiter.

Zähle von 1 bis 5 hoch:

Beispiel

```
zaehler = 1
while zaehler <= 5:
    print(zaehler)
    zaehler += 1
```

So läuft eine while-Schleife **Schritt für Schritt** ab:

1. Die **Bedingung** hinter while wird geprüft.
2. Ist sie True, wird der **eingerrückte Block** ausgeführt.
3. Danach springt Python **zurück** zum while und prüft die Bedingung erneut.
4. Das wiederholt sich, bis die Bedingung False ergibt.

Die Bedingung ist – wie bei if – immer ein **Wahrheitswert** (bool): entweder True oder False.

Achtung, Endlosschleife! Vergisst du `zaehler += 1`, bleibt die Bedingung für immer True und das Programm läuft ewig weiter. **Merke:** In jeder while-Schleife muss sich etwas verändern, das die Bedingung irgendwann False werden lässt.

Aufgabe 1: Was passiert, wenn man in der while-Schleife `zaehler += 1` vergisst?

- ☐ a) Die Schleife läuft genau einmal
- ☐ b) Die Schleife läuft endlos weiter (Endlosschleife)
- ☐ c) Python meldet automatisch einen Fehler



Ein anschauliches Beispiel: **Lucy lernt laufen**. Man geht davon aus, dass Lucy laufen kann, sobald sie 100 Schritte am Stück geschafft hat. Bei jedem Versuch (Schleifendurchlauf) geht sie 2 Schritte mehr:

Beispiel

```
schritte = 0
while schritte < 100:
    schritte += 2
    print(f"Lucy läuft: {schritte} Schritte")
print("Lucy kann jetzt laufen!")
```

Aufgabe 2: Schreibe eine while-Schleife, die von 5 rückwärts bis 1 zählt und jede Zahl in einer eigenen Zeile ausgibt.

Dein Code:

Sehr häufig braucht man eine Schleife, um **Werte aufzusummieren**. Dazu legt man vor der Schleife eine Variable `summe = 0` an und addiert in jedem Durchlauf einen Wert hinzu. Man nennt so eine Variable auch **Summierer** (oder Akkumulator).

Aufgabe 3: Berechne mit einer while-Schleife die Summe aller Zahlen von 1 bis 10 (also $1 + 2 + \dots + 10$) und gib nur das Ergebnis aus.

Erwartet: **55**

Dein Code:



while-Schleife anwenden

Jetzt kombinierst du die `while`-Schleife mit einer **Eingabe**. Der Benutzer gibt eine Zahl ein, und die Schleife arbeitet mit dieser Zahl weiter. So werden Schleifen erst richtig nützlich.

Denk daran: `input()` liefert immer **Text** – für Rechnungen wandelst du ihn mit `int(...)` in eine ganze Zahl um.

Ein Countdown, der bei einer eingegebenen Zahl startet:

Beispiel

```
zahl = int(input("Start: "))
while zahl >= 1:
    print(zahl)
    zahl -= 1
print("Los!")
```

Aufgabe 4: Frage mit `zahl`: eine ganze Zahl ab. Gib dann alle Zahlen von dieser Zahl herunter bis 1 aus – jede in einer eigenen Zeile.

Beispiel: Eingabe **5** → 5, 4, 3, 2, 1 (untereinander).

Dein Code:



Als Nächstes ein **Summierer**: Der Benutzer gibt eine Zahl n ein, und das Programm addiert alle Zahlen von 0 bis n .

Dazu läuft eine Hilfsvariable i von 0 hoch bis n und wird jedes Mal auf summe addiert.

Aufgabe 5: Frage mit Zahl1: eine ganze Zahl n ab. Berechne die Summe aller ganzen Zahlen von 0 bis n und gib sie aus.

Beispiel: Eingabe **4** → **10** (denn $0 + 1 + 2 + 3 + 4 = 10$).

Dein Code:

Zum Schluss die **Fakultät**. Die Fakultät einer Zahl ist das **Produkt** aller Zahlen von 1 bis zu dieser Zahl. Beispiel: $4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$.

Hier brauchst du keinen Summierer, sondern einen **Produkt-Sammler**: Er startet bei 1 (nicht 0!) und wird mit $\cdot$ multipliziert. **Wichtig**: Die Fakultät von 0 ist per Definition 1.

Aufgabe 6: Frage mit Zahl1: eine ganze Zahl ab und berechne ihre Fakultät (das Produkt aller Zahlen von 1 bis zu dieser Zahl). Gib das Ergebnis aus.

Beispiel: Eingabe **4** → **24** ($4 \cdot 3 \cdot 2 \cdot 1$). Eingabe **0** → **1**.

Dein Code: